אוניברסיטת בן-גוריון
מדור בחינות

מספר נבחן:_____

תאריך הבחינה:	12.5.2024
שם המורה	ד"ר מני אדלר יאיר ועקנין
שם הקורס:	תכנות מערכות
מספר הקורס:	202-1-2031
שנה:	תשפ"ד
סמסטר:	א
מועד:	ב
משך הבחינה:	שעתיים
חומר עזר:	אין

רשמו תשובותיכם בגיליון התשובות בלבד, תשובות מחוץ לגיליון לא יבדקו.
ניתן לענות 'לא יודע' על סעיף שלם (בלבד) ולקבל 20% מהניקוד עבורו

בהצלחה!

שאלה 1: זיכרון (35 נקודות)

האופטימיזציה 'copy on write' משהה את השכפול של אובייקט עד אשר תתבצע פעולה המשנה אותו. האובייקט לא ישוכפל כעת: כל עוד רק קוראים ממנו האובייקט יהיה משותף, ורק כאשר יתבצע בו שינוי כלשהו ייווצר עותק.

לדוגמא, בקטע הקוד למטה נרצה לחסוך את השכפול של a: a ו-b יצביעו לאותו אובייקט, כי הוא לא השתנה:

```
class A {
private:
    int i;

public:
    A(int i) : i(i) {}
    A* clone() { return new A(*this); }
    int get() { return i; }
    void set(int j) { i = j; }

};

{
    A* a = new A(3);
    A* b = a->clone();
    std::cout << a->get();
    std::cout << b->get();
}
```

לעומת זאת, בקטע הקוד הבא נרצה ש-a ו-b יצביעו לאובייקטים שונים כי בוצעה השמה על אחד מהם:

```
{
    A* a = new A(3);
    A* b = a->clone();
    std::cout << b->get();
    a->set(5);
}
```

כדי לממש את האופטימיזיה, נגדיר את התבנית copy_on_write.
המחלקה עוטפת מצביע לאובייקט מטיפוס T (ניתן להניח בשאלה שהוא מממש מתודת clone) הניתן בבנאי. מי שקורא למתודה on_write מציין כי הוא הולך לבצע לראשונה פעולת השמה על האובייקט T, כך שנדרש לשכפלו.

א. השלימו את מימוש המחלקה copy_on_write

```
template <typename T>
class copy_on_write {

public:
    T* ptr;

    copy_on_write(T* p) : ptr(p) {}

    void on_write() {
        //@@TODO
    }
};
```

[5 נקודות]

ב. עדכנו את קטע הקוד הבא כך שהאופטימיזציה תתרחש, וציירו את תמונת הזיכרון במקומות המצויינים:

```
 {
    A* a = new A(3);
    A* b = a->clone();
```

```
    //@@Draw the memory
    std::cout << b->get();
    a->set(5);
    //@@Draw the memory
}
```

[15 נקודות]

ג. הסבירו מדוע בקטע הקוד הבא לא חייבים לשכפל את האובייקט, ותקנו/הרחיבו את הקוד המחלקה copy-on-write כך ששכפול האובייקט ייעשה רק כאשר זה אכן נדרש.
[15 נקודות]

```
{
    A* a = new A(3);
    {
       A* b = a->clone();
       std::cout << a->get();
       std::cout << b->get();
    }
    a->set(5);
    ...
}
```

שאלה 2: מקביליות (30 נקודות)

נתונה המחלקה SetImpl:

```
public interface Set<T> {
    boolean contains(T obj);
    void add(T obj);
    Iterator<T> iterator();
    Set<T> clone();
}

class SetImpl<T> implements Set<T> {

    private volatile Link<T> head;

    SetImpl() {
        head = null;
    }

    SetImpl(Link<T> head) {
        this.head = head;
    }

    public boolean contains(T obj) {
        if (head == null)
            return false;
        else
            return contains(head,obj);
    }

    private boolean contains(Link<T> curr, T obj) {
        if (head.data.equals(obj))
            return true;
        if (head.next == null)
            return false;
```

```
        return contains(head.next, obj);
    }

    public void add(T obj) {
        if (head == null)
            head = new Link<T>(obj,null);
        else
            add(head,obj);
    }

    private void add(Link<T> curr, T obj) {
        if (curr.data.equals(obj))
            return;
        if (curr.next == null)
          curr.next = new Link(obj,null);
        else
            add(curr.next,obj);
    }

    public Iterator<T> iterator() {
        return new Iterator() {
            private Link<T> next = head;

            public boolean hasNext() {
                return next != null;
            }

            public T next() {
                T ret = next.data;
                next = next.next;
                return ret;
            }

            public void remove() {}
        };
    }
```

```
    public Set<T> clone() {
        if (head == null)
            return new SetImpl();
        else {
            Link<T> this_curr_link = head;
            Link<T> copy_curr_link =
                   new Link<T>(this_curr_link.data,null);
            Set<T> ret = new SetImpl(copy_curr_link);
            while (this_curr_link.next != null) {
                copy_curr_link.next =
                    new Link<T>(this_curr_link.next.data,null);
                this_curr_link = this_curr_link.next;
                copy_curr_link = copy_curr_link.next;
            }
            return ret;
        }
    }

    private static final class Link<T> {
        public data;
        public Link<T> next;

        Link(T data, Link<T> next) {
            if (data == null)
               throw new Exception("Null Data");
            this.data = data;
            this.next = next;
        }

    }
}
```

א. סנכרנו את המחלקה כך שהיא תהיה בטוחה ונכונה בחישוב מקבילי. אין צורך להעתיק קוד, ציינו רק מה נדרש לעשות.
[5 נקודות]

ב. ממשו את המחלקה בתבנית Immutable Objects
[10 נקודות]

ג. האם הייתם ממליצים להשתמש במחלקה שמימשתם בסעיף א או במחלקה שמימשתם בסעיף ב עבור תוכנית מקבילית? נמקו בקצרה תוך התייחסות לסיבות שונות (ניתן לענות על הסעיף גם אם לא מימשתם את סעיף ב)
[5 נקודות]

ד. למחלקה המסונכרנת מסעיף א נוספה המתודה **printAll**.
להלן שתי אפשרויות מימוש למתודה:

```
public void printAll() {
    Iterator<T> it = iterator();
    while (it.hasNext())
        System.out.println(it.next());
}
```

```
public void printAll() {
   Iterator<T> it = clone().iterator();
   while (it.hasNext())
       System.out.println(it.next());
}
```

- האם קיים תרחיש בעייתי באחד המימושים עבור הרצה מקבילית של המחלקה הנתונה?
- האם ההדפסה בשני המימושים תהיה זהה?
- באיזה מהמימושים הייתם משתמשים?

[10 נקודות]

שאלה 3: תקשורת ובסיסי נתונים (35 נקודות)

בנספח (בסוף השאלון), מופיע לנוחיותכם קוד ה-BaseServer כפי שנלמד בכיתה.

א. עדכנו את קוד תבנית השרת כך שהלקוח יכול לשלוח הודעה מטיפוס אחד ולקבל תשובה מטיפוס שני (כלומר השרת מקבל הודעה מטיפוס אחד ומחזירה תשובה מטיפוס אחר)

אין צורך לשכתב מחלקות שלמות - ציינו רק את המקומות שבהם אתם עורכים שינוי בקוד, ואת הקוד של השינוי.

[10 נקודות]

ב. השלימו את הגדרת הממשק **Command** ואת מימוש המחלקה **RemoteCommandInvocationProtocol** עבור שרת הפקודות, עבור התבנית המעודכנת מסעיף א (ניתן לענות גם אם לא מימשתם את סעיף א). אין להשתמש ב-casting.

```
public interface Command<T> extends Serializable {
    _________ execute(T data);
}

public class RemoteCommandInvocationProtocol<T>
     implements MessagingProtocol<_________> {

    private T data;

    public RemoteCommandInvocationProtocol(T data) {
        this.data = data;
    }

    @Override
    public _____________ process(____________________) {
        ___________________________________________
    }

    @Override
    public boolean shouldTerminate() {
        return false;
    }
}
```

[10 נקודות]

נתונה המחלקה **DB** ב-**Java** המאפשרת ביצוע שאילתות על בסיס נתונים רלציוני.
אופן מימוש המחלקה אינו רלבנטי לשאלה זו. נתון כי היא בטוחה ונכונה בריצה מקבילית, וכי יש לה בנאי ריק.
המתודה המרכזית במחלקה היא **executeQuery**:

```
public ArrayList<String> executeQuery(String query);
```

המתודה מקבלת שאילתא, מריצה אותה על בסיס הנתונים, ומחזירה את מערך הרשומות שהשאילתא החזירה
(מימוש המחלקה אינו רלבנטי לשאלה)

ג. השלימו את התוכנית **DBServer** המריצה שרת פקודות על אובייקט **DB,** מבוסס BaseServer עם מאגר קבוע של 11 ת'רדים.

תזכורת: למחלקה שממממשת MessageEncoderDecoder<Serializable> קוראים ObjectEncoderDecoder ויש לה בנאי ריק.

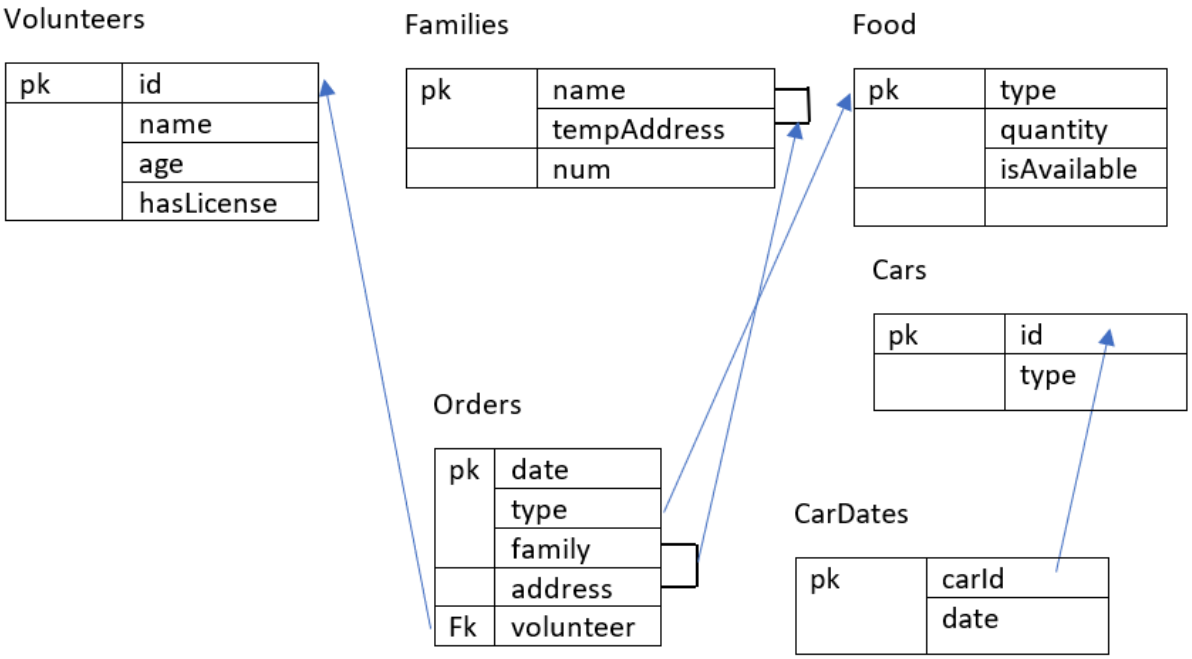
```
...
class DBServer {
    public static void main(String[] args) {
        int port = IntegerParseInt(args[0]);
        ...
    }
}
```

[5 נקודות]

במסגרת ייעול המחסן הלוגיסטי עבור האזרחים המפונים, הוחלט להקים בסיס נתונים שירכז את כל המידע הנדרש לפעילות זו:

- מתנדב: מ.ז., שם, גיל, האם יש רשיון נהיגה בתוקף?
- משפחה מפונה: שם משפחה, כתובת זמנית (כמחרוזת), מספר נפשות (נתון כי אין משפחות בעלות שם זהה באותה כתובת זמנית, אין 'מספר מזהה למשפחה')
- מנת מזון: סוג מנה, כמות מנות זמינות כרגע
- רכב: מספר רכב, סוג רכב, תאריכים שבהם הרכב זמין לשימוש
- הזמנה: תאריך, סוג מנה, משפחה, מתנדב אחראי.

זהו מודל הנתונים שנבחר:



ד. השלימו את מתודת ה-main במחלקה DBClient (ניתן להגדיר מחלקה נוספת), כך שהוא תשלח בקשה להרצת שאילתת SQL המחזירה את ההזמנות שהמתנדב שאחראי עליהן יכול לנהוג ברכב, ממוינות ע"פ תאריך ההזמנה בסדר עולה. בתוצאות יש לכלול את: תאריך ההזמנה, סוג המנה, שם המשפחה אליה נשלחת המנה, שם המתנדב האחראי.
תוכנית הלקוח ממתינה לתשובה ומדפיסה את התוצאות עם הגעתן.
ניתן להוסיף מחלקה למערכת.

```
public class RCIClient implements Closeable {

    private final ObjectEncoderDecoder encdec;
    private final Socket sock;
    private final BufferedInputStream in;
    private final BufferedOutputStream out;

    public RCIClient(String host, int port) throws IOException {
        sock = new Socket(host, port);
        encdec = new ObjectEncoderDecoder();
        in = new BufferedInputStream(sock.getInputStream());
        out = new BufferedOutputStream(sock.getOutputStream());
    }
```

```
    public void send(Command<?> cmd) throws IOException {
        out.write(encdec.encode(cmd));
        out.flush();
    }

    public Serializable receive() throws IOException {
        int read;
        while ((read = in.read()) >= 0) {
            Serializable msg = encdec.decodeNextByte((byte) read);
            if (msg != null) {
                return msg;
            }
        }

        throw new IOException("disconnected before complete reading message");
    }

    @Override
    public void close() throws IOException {
        out.close();
        in.close();
        sock.close();
    }
}

public class DBClient {

    public static void main(String[] args) throws Exception {
       RCIClient c = new RCIClient(args[0], 7777);
       ...
       ...
    }
}
...
...
```

[10 נקודות]

נספח: קוד ה-BaseServer, כפי שנלמד בכיתה

```
public interface MessageEncoderDecoder<T> {
    T decodeNextByte(byte nextByte);
    byte[] encode(T message);
}

public interface MessagingProtocol<T> {
    T process(T msg);
    boolean shouldTerminate();
}


public class ConnectionHandler<T> implements Runnable {

    private final MessagingProtocol<T> protocol;
    private final MessageEncoderDecoder<T> encdec;
    private final Socket sock;

    public ConnectionHandler(Socket sock, MessageEncoderDecoder<T> reader,
               MessagingProtocol<T> protocol) {
        this.sock = sock;
        this.encdec = reader;
        this.protocol = protocol;
    }

    @Override
    public void run() {

        try (   Socket sock = this.sock; //just for automatic closing
                BufferedInputStream in =
                   new BufferedInputStream(sock.getInputStream());
                BufferedOutputStream out =
                   new BufferedOutputStream(sock.getOutputStream())) {

            int read;
            while (!protocol.shouldTerminate() && (read = in.read()) >= 0) {
                T nextMessage = encdec.decodeNextByte((byte) read);
                if (nextMessage != null) {
                    T response = protocol.process(nextMessage);
                    if (response != null) {
                        out.write(encdec.encode(response));
                        out.flush();
                    }
                }
            }
```

```
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}

public abstract class BaseServer {

    private final int port;
    private final Supplier<MessagingProtocol> protocolFactory;
    private final Supplier<MessageEncoderDecoder> encdecFactory;

    public BaseServer(
            int port,
            Supplier<MessagingProtocol> protocolFactory,
            Supplier<MessageEncoderDecoder> encdecFactory) {

        this.port = port;
        this.protocolFactory = protocolFactory;
        this.encdecFactory = encdecFactory;
    }

    public void serve() {
        try (ServerSocket serverSock = new ServerSocket(port)) {

            while (!Thread.currentThread().isInterrupted()) {

                Socket clientSock = serverSock.accept();
                ConnectionHandler handler = new ConnectionHandler(
                        clientSock,
                        encdecFactory.get(),
                        protocolFactory.get());

                execute(handler);
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }

        System.out.println("server closed!!!");
    }


    protected abstract void execute(ConnectionHandler handler);
}


public class SingleThreadedServer extends BaseServer {
```

```
    public SingleThreadedServer(
            int port,
            Supplier<MessagingProtocol> protocolFactory,
            Supplier<MessageEncoderDecoder> encoderDecoderFactory) {

        super(port,protocolFactory,encoderDecoderFactory);
    }

    @Override
    protected void execute(ConnectionHandler handler) {
        handler.run();
    }

}


public class ThreadPerClientServer extends BaseServer {

    public ThreadPerClientServer(
            int port,
            Supplier<MessagingProtocol> protocolFactory,
            Supplier<MessageEncoderDecoder> encoderDecoderFactory) {

        super(port, protocolFactory, encoderDecoderFactory);
    }

    @Override
    protected void execute(ConnectionHandler handler) {
        new Thread(handler).start();
    }

}


public class FixedThreadPoolServer extends BaseServer {

    private final ExecutorService pool;

    public FixedThreadPoolServer(
            int numThreads,
            int port,
            Supplier<MessagingProtocol> protocolFactory,
            Supplier<MessageEncoderDecoder> encoderDecoderFactory) {

        super(port, protocolFactory, encoderDecoderFactory);
        this.pool = Executors.newFixedThreadPool(numThreads);
    }
```

```
    @Override
    public void serve() {
        super.serve();
        pool.shutdown();
    }

    @Override
    protected void execute(ConnectionHandler handler) {
        pool.execute(handler);
    }
}
```